BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 2

Roll no: 11 PRN No: 12413071 Name of Student: Patil Shreyas Prashant

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1) Start the program.

2) Import the required libraries:

i. Numpy
ii. Matplotlib
iii. Pandas
iv. Seaborn
v. Scikit-learn
vi. TensorFlow/Keras

3) Load the Iris dataset (Iris.csv) into a Pandas DataFrame.Separate the dataset into:

i. Training data
ii. Testing data

4) Display the dataset information such as:

i. Shape of the dataset
ii. Column names
iii. First five records

5) Preprocess the dataset:

i. Remove the Id column (if present).
ii. Separate the input features (X**)** and target labels (y).
iii. Convert categorical target labels into numerical values using LabelEncoder.

6) Split the dataset into training and testing sets using an 80:20 ratio.

7) Normalize the input features using StandardScaler.

8) Train the MLP model using the training dataset for 50 epochs with a batch size of 8.

9) Evaluate the trained model using the testing dataset.

10) Visualize the Confusion Matrix using a heatmap.

Code :

[25] ✓ 0s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

[2] ✓ 0s

```
df = pd.read_csv("Iris.csv")

df.head()
```

[3] ✓ 0s

```
print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nDataset Information:")
df.info()

print("\nFirst Five Rows:")
print(df.head())
```

[4] ✓ 0s

```
print(df.isnull().sum())
```

[12] ✓ 0s

```
X = df.iloc[:, 1:5]
y = df.iloc[:, 5]

print(X.head())
print(y.head())
```

[14] ✓ 0s

```
encoder = LabelEncoder()

y = encoder.fit_transform(y)

print("Classes:")
print(encoder.classes_)
```

[15] ✓ 0s

```
print("Feature Shape:", X.shape)
print("Target Shape:", y.shape)
print("Feature Columns:")
print(X.columns)
```

[16] ✓ 0s

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

[17] ✓ 0s

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[18] ✓ 0s

```
model = Sequential()

model.add(Dense(16, activation='relu', input_shape=(4,)))
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
```

[19] ✓ 0s

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

[20] ✓ 9s

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.2,
    verbose=1
)
```

[21] ✓ 0s

```
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)
```

[22] ✓ 0s

```
y_pred = model.predict(X_test)

y_pred_classes = np.argmax(y_pred, axis=1)
```

[23] ✓ 0s

```
acc = accuracy_score(y_test, y_pred_classes)

print("Accuracy Score:", acc)
```

[26] ✓ 0s

```
print("Classification Report:\n")

print(classification_report(
    y_test,
    y_pred_classes,
    target_names=encoder.classes_
))
```

[27] ✓ 0s

```
cm = confusion_matrix(y_test, y_pred_classes)

print("Confusion Matrix:\n")
print(cm)
```

[28] ✓ 0s

```
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=encoder.classes_,
    yticklabels=encoder.classes_
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

[29] ✓ 0s

```
plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()

plt.show()
```

Output :

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
Dataset Shape: (150, 6)

Column Names:
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB

First Five Rows:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm      Species
0   1            5.1           3.5            1.4           0.2  Iris-setosa
1   2            4.9           3.0            1.4           0.2  Iris-setosa
2   3            4.7           3.2            1.3           0.2  Iris-setosa
3   4            4.6           3.1            1.5           0.2  Iris-setosa
4   5            5.0           3.6            1.4           0.2  Iris-setosa
```

```
Id               0
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64
```

```
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
0            5.1           3.5            1.4           0.2
1            4.9           3.0            1.4           0.2
2            4.7           3.2            1.3           0.2
3            4.6           3.1            1.5           0.2
4            5.0           3.6            1.4           0.2
0    Iris-setosa
1    Iris-setosa
2    Iris-setosa
3    Iris-setosa
4    Iris-setosa
Name: Species, dtype: object
```

```
Classes:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

```
Feature Shape: (150, 4)
Target Shape: (150,)
Feature Columns:
Index(['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'], dtype='object')
```

```
Training Samples: 120
Testing Samples: 30
```

```
Accuracy Score: 1.0
```

```
Classification Report:

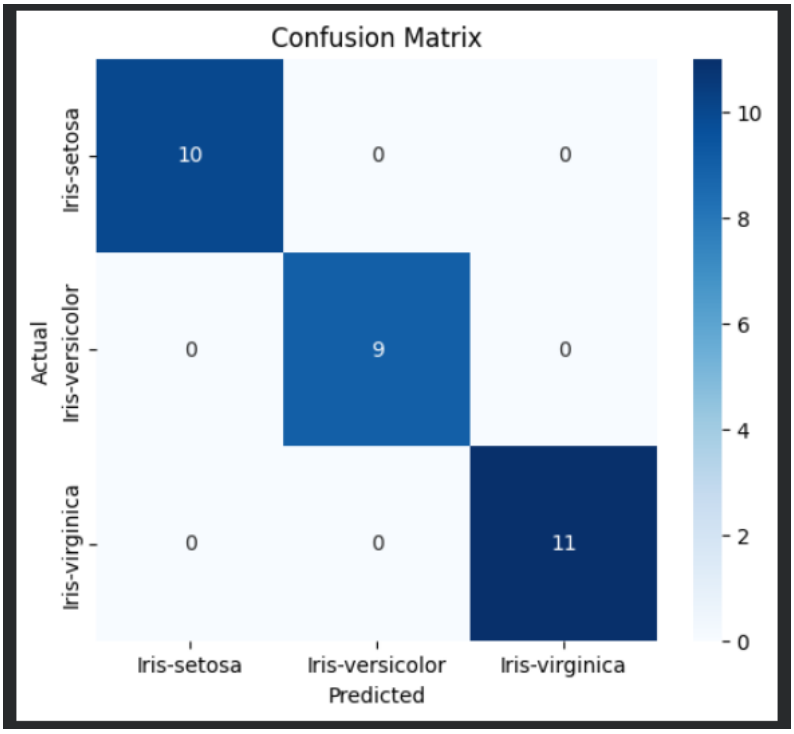
                 precision    recall  f1-score   support

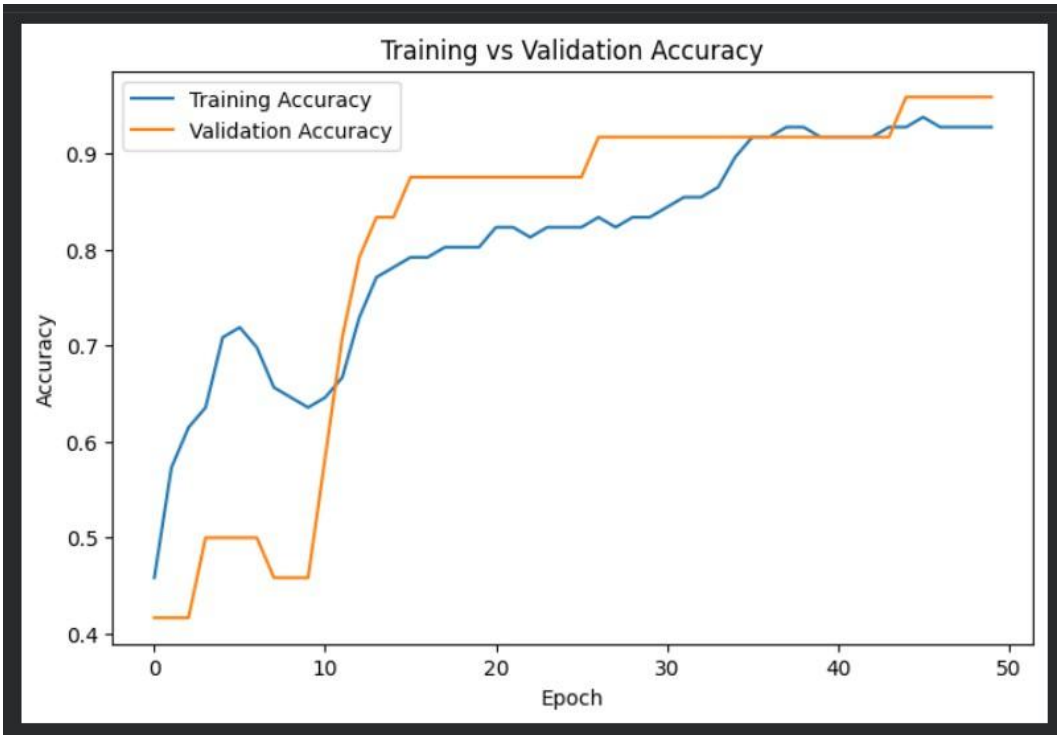
    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

```
Confusion Matrix:

[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```


Confusion Matrix
Iris-setosa
Iris-versicolor
Iris-virginica
Actual
10
0
0
0
9
0
0
0
11
Iris-setosa
Iris-versicolor
Iris-virginica
Predicted
10
8
6
4
2
0


Training vs Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
0.9
0.8
0.7
0.6
0.5
0.4
0
10
20
30
40
50
Epoch